Year 4 Project
[CSC1097]

Tag Battle Royale

[Testing Documentation]

Project Team:	Luan Slankster	luan.slankster2 @mail.dcu.ie	22406536
	Michael Cojocaru	michael.cojocaru2 @mail.dcu.ie	22380876
Supervisor:	Darragh O'Brien	darragh.obrien @dcu.ie	

1st May 2026

Table of Contents

1. Unit Testing..... 3
1.1 Face Detection..... 3
1.2 Map File Handling..... 4
1.3 Session Persistence..... 4
2. Manual System Testing..... 5
2.1 Face Detection..... 5
2.2 Offline Connectivity..... 10
2.2.1 Map Payloads..... 10
2.2.2 Session Restoration..... 12
2.3 Disposability..... 14
3. Manual Integration Testing..... 17
3.1 Online Connectivity..... 17
3.1.1 Connection Loss..... 17
3.1.2 Session Resumption..... 18
4. Manual Functional Testing..... 20
4.1 Maps..... 20
4.2 Game..... 21
4.3 Replay Viewer..... 24
5. User Testing..... 25
5.1 Introduction..... 25
5.2 Methods..... 25
5.3 Results..... 25
5.3.1 Participant Data..... 25
5.3.2 Map Editor..... 26
5.3.3 Offline Mode..... 27
5.3.4 Online Mode..... 28
5.3.5 Replays..... 29
5.3.6 Overview..... 30
5.4 Conclusion..... 31
6. Appendix..... 32

1. Unit Testing

Unit tests ensure consistent functionality throughout iterations. These tests were integrated with Gitlab's built-in CI pipelines, which allowed for quick set up and easy analysis, and were run for every push. We implemented our unit tests with the flutter_test library due to its accessibility. They are located within 2026-csc1097-tag-battle-royale/src/tag_battle_royale/test. Due to lack of complete inputs and the difficulty in simulating tests that involve real hardware, more thorough testing was conducted in manual tests.

1.1 Face Detection

face_detection_test.dart

When conducting the manual tests for face detection, the inputs for tricky situations that had caused errors were captured. These inputs were used in the unit tests, to ensure these edge case situations were being handled correctly. Additionally testCase1 is a more simple situation that addresses a more general case for the face detector. These inputs are around 100 - 200 points.

For each test case the strokes are added to the face detector and the graph is built. Each test case has an expected number of faces which is used to determine if the test passes or fails. This was designed in a way that any future cases that are found can be easily inserted.

1. **Description:** Square with a cross through it and a circle on top
   Expected: 5 faces
   Result: pass

2. **Description:** Square then 3/4 square from top middle to right middle and a line through right middle connection
   Expected: 2 faces
   Result: pass

3. **Description:** Tests endpoint snap tolerance, two square squares sharing a side with a line crossing the top right of the right square
   Expected: 3 faces
   Result: pass

4. **Description:** Duplicate edge key, rectangle with two triangles sharing a side
   Expected: 3 faces
   Result: pass

5. **Description:** Bridge from one enclosed region to another, with one region contained in the other
   Expected: 2 faces
   Result: pass

1.2 Map File Handling

map_file_handler_test.dart

To ensure consistent mapfile behaviour, multiple unit tests were implemented. These tests covered correct conversion to json and from json behaviour, map file saving, map file retrieving and finally map file deletion.

1. **Description:** MapFile toJson and fromJson
   Expected: Test map is correctly rebuilt
   Result: pass

2. **Description:** saveMapFile and getMaps
   Expected: Correct map is successfully saved and retrieved
   Result: pass

3. **Description:** delete removes file
   Expected: MapFile is successfully removed
   Result: pass

1.3 Session Persistence

session_persistence.dart

Multiple tests were composed to see if the project would successfully convert session files to and from json including map and replay.

Offline

1. **Description:** round-trip preserves all fields
   Expected: All fields are present
   Result: pass

2. **Description:** empty JSON uses defaults
   Expected: Default fields present
   Result: pass

Online

1. **Description:** round-trip preserves all fields
   Expected: All fields are present
   Result: pass

2. **Description:** empty JSON uses defaults
   Expected: Default fields present
   Result: pass

2. Manual System Testing

2.1 Face Detection

The following tests helped understand and verify the face detection logic for the map editor.

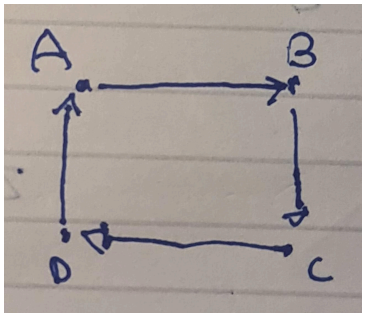
Planned Tests

For the following tests the following structure was followed:

1. A test was designed and an expected number of faces was determined
2. The test was conducted and the actual faces found by the face detection logic was recorded.
3. If the number of expected faces and actual faces did not match then the cause of this discrepancy was investigated, a change was made and the test was repeated.
4. This information was recorded with pen and paper and has been included below.

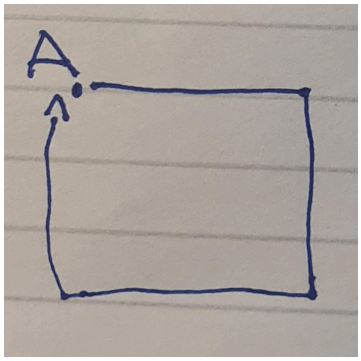
Test 1

Stage	Expected	Actual
Initial	1	0
Repeat	1	1
Reversed	1	1



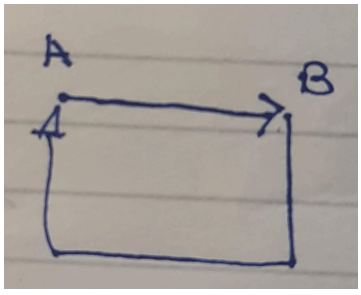
Test 2

Stage	Expected	Actual
Initial	1	1
Repeat	1	1
Reversed	1	1



Test 3

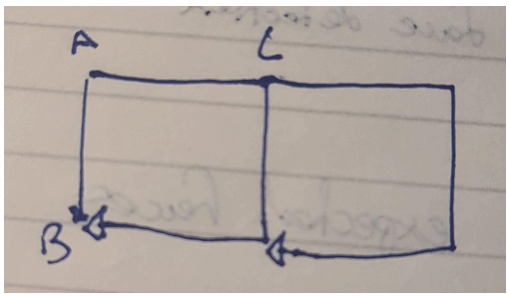
Stage	Expected	Actual
Initial	1	0
Repeat	1	1
Reversed	1	1



Changes Made 1-3:
Tests 1, 2 and 3 were similar test cases used to evaluate the initial stages of the system. During these tests the endpointToSegmentTolerance was adjusted from 7 to 15.

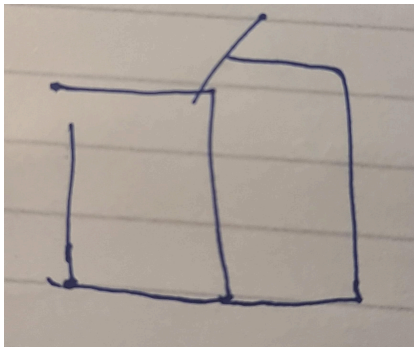
Test 4

Stage	Expected	Actual
Initial	2	2
Reversed	2	2



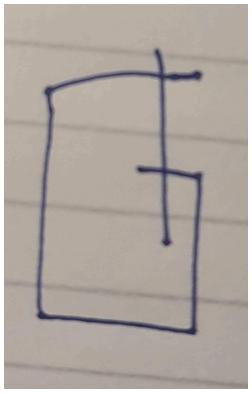
Test 5

Stage	Expected	Actual
Initial	1	0
Repeat	1	1



Test 6

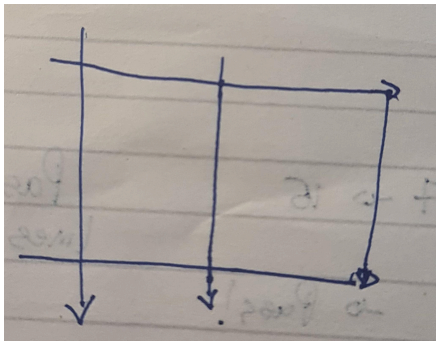
Stage	Expected	Actual
Initial	1	0
Repeat	1	1



Changes Made 4-6:
The issue found in tests 5 and 6 was the same. In both cases, the strokes used to draw the faces were having segment to segment intersections and caused hanging edges to be included. These dead ends were causing the graph representing the faces to no longer be planar and the face detection logic to no longer work correctly. To address this a graph pruning step was added to remove dead ends and ensure the graph is planar.

Test 7

Stage	Expected	Actual
Initial	2	1
Repeat	2	1



Changes Made 7:
For test 7 the endpoint to endpoint logic was operating incorrectly when a slight gap was introduced. To address this the endpoint to endpoint intersection logic was improved

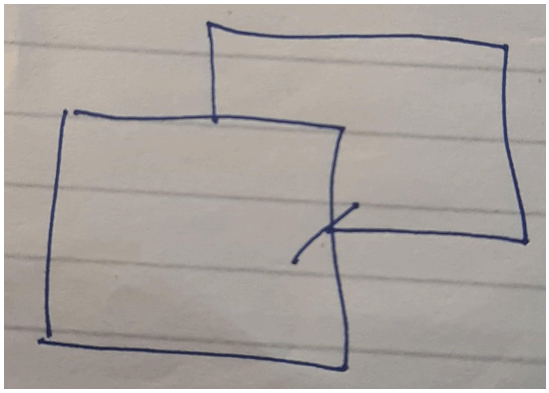
Discovered Tests
The following test cases were not pre-planned and were found through a more informal process, defined as follows:

1. Different shapes and ordering of shapes were drawn at random.
2. When behaviour wasn't as expected, the inputs used to draw the shapes were printed to the terminal.
3. The test case was investigated and a change was implemented.
4. The inputs recorded were used to investigate the test case again.

The following "discovered tests" were the most important in finding bugs in the face detection logic. Recording the inputs allowed confidence that the issue was actually solved instead of a different drawing simply avoiding the issue. The following Images were originally recorded via screenshot but unfortunately the home address of a member of the development team was included. Instead of the original screenshots, recreations and obfuscations were used.

Test 8

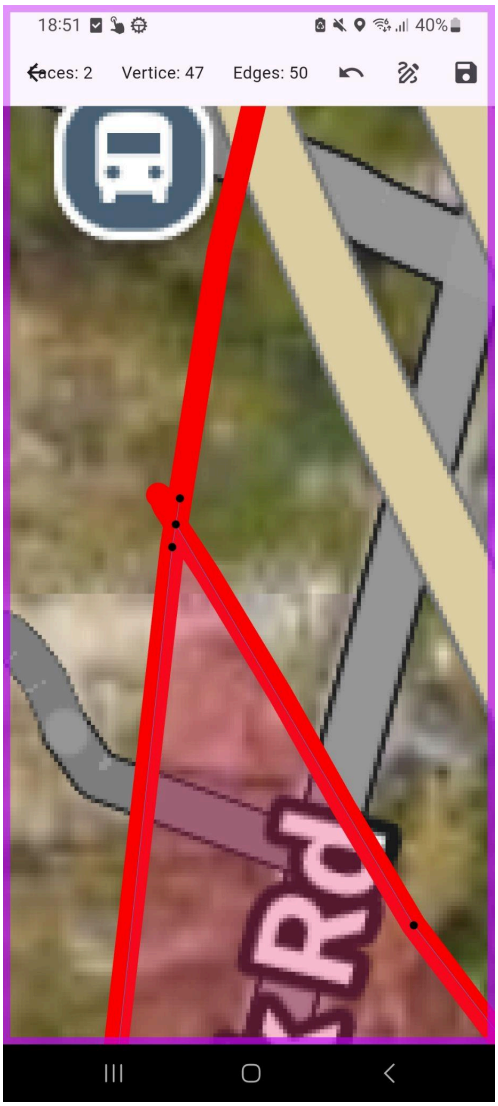
Stage	Expected	Actual
Initial	2	1
Repeat	2	2



Changes Made 8:
For Test 8 a square was drawn with another ¾ square connecting to the original squares top center and right center points. At this point behaviour was as expected. Only sometimes, when a line was drawn through the intersection point at the right middle, one of the two faces would no longer be found. Which face was no longer found would depend on the location of the additional intersecting line. This was caused by the endpoint segment intersection code incorrectly recording the intersections, resulting in the connection being severed. The solution included recording all relevant segments for this intersection.

Test 9

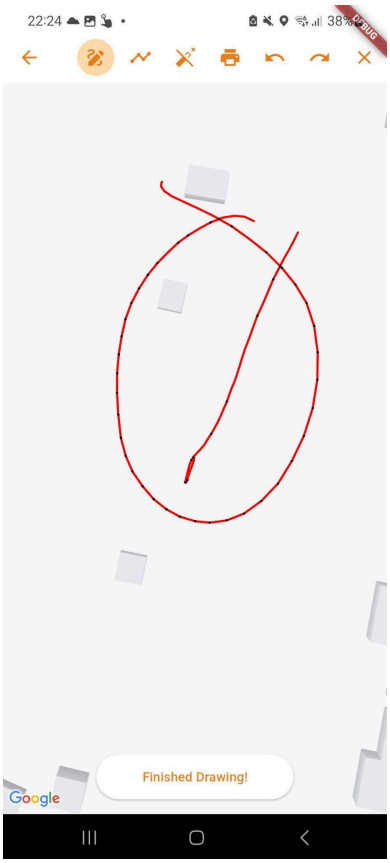
Stage	Expected	Actual
Initial	3	2
Repeat	3	3



Changes Made 9:
For test 9, within a larger shape when drawing a triangle, an intersection was being incorrectly pruned. As seen in the image, there is an additional vertex from the endpoint being projected onto the segment that is incorrectly being included in the planar graph. This issue was caused by the edge from the edge of the corner of the triangle and the dead end being included twice. To address this issue a unique edge key method was included. Now V11 -> V12 and V12 -> V11 will produce the same key Vsmaller -> Vbigger.

Test 10

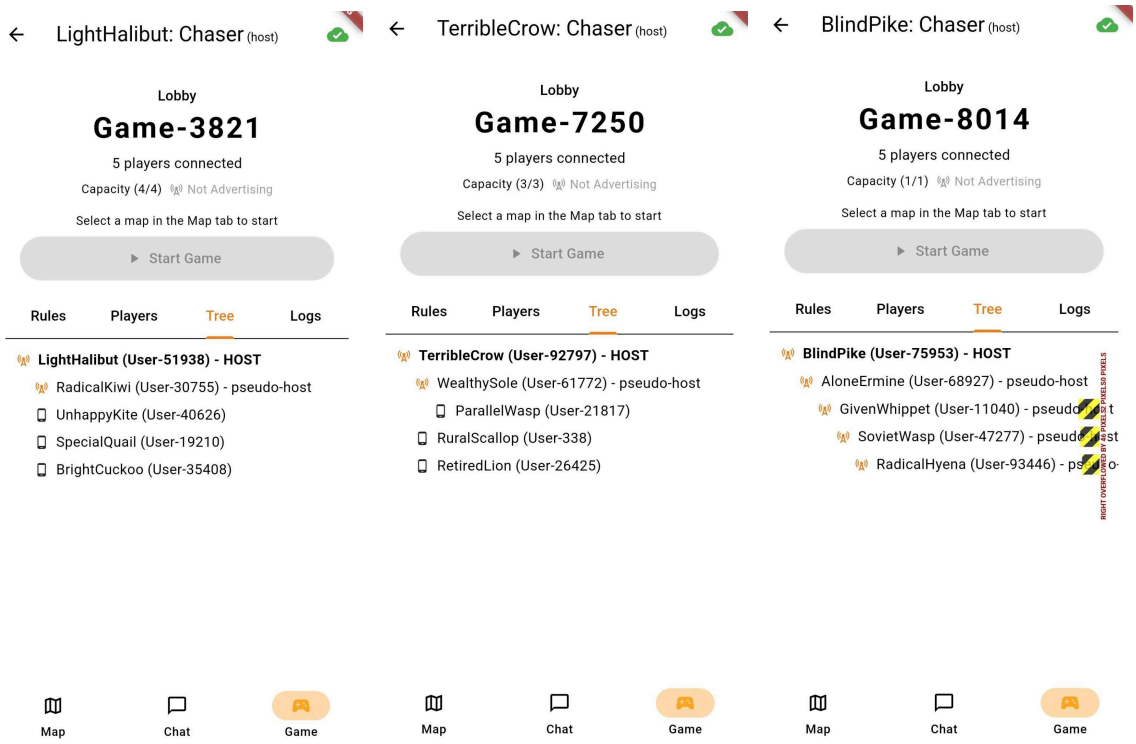
Stage	Expected	Actual
Initial	1	0
Repeat	1	1



Changes Made 10:
Test 10 was found during user testing and is caused by a bridge connecting an outer face to an inner face contained within the bigger face. This bridge breaks the planarity of the graph and disrupts the face tracing. A new step to find any bridges was implemented using Tarjan's bridge-finding algorithm

2.2 Offline Connectivity

The network made with Nearby Connections was tested in three different configurations with five devices. The max capacity of devices were changed between tests to accomplish these configurations. The configurations are listed below:



- **Configuration 1:**
  maxConnections = 4
  Each device connected to host, one empty pseudo-host.

- **Configuration 2:**
  maxConnections = 3
  Three devices connected to host, one device connected to pseudo-host.

- **Configuration 3:**
  maxConnections = 1
  Each device connected in a chain from host, all non-hosts are pseudo-hosts with one child, barring the last in the chain.

2.2.1 Map Payloads

These tests verify if map payloads of different sizes are delivered correctly to all devices. Maps typically exceed the 32kb limit and require chunking. Included are the times it took for the tests to complete, to test the performance of the configurations.

Configuration 1 (maxConnections = 4, flat tree)

Test	Map Size	Time to Complete	Result
1	~307 KB (10 chunks)	4.98s	pass
2		3.82s	pass
3		4.76s	pass
4		6.22s	pass
1	~1107 KB (35 chunks)	15.72s	pass
2		17.70s	pass
3		15.98s	pass
4		16.62s	pass

Configuration 2 (maxConnections = 3, one pseudo-host)

Test	Map Size	Time to Complete	Result
1	~307 KB (10 chunks)	5.31s	pass
2		4.04s	pass
3		5.02s	pass
4		6.58s	pass
1	~1107 KB (35 chunks)	16.14s	pass
2		17.23s	pass
3		15.85s	pass
4		16.91s	pass

Configuration 3 (maxConnections = 1, chain)

Test	Map Size	Time to Complete	Result
1	~307 KB (10 chunks)	8.34s	pass
2		10.42s	pass
3		6.42s	pass
4		8.34s	pass
1	~1107 KB (35 chunks)	34.14s	pass
2		28.81s	pass
3		38.46s	pass
4		26.22s	pass

Observations:
Times increase with each configuration as expected.

- Configuration 1 delivers directly to all devices.
- Configuration 2 adds one relay hop for one device which allows for parallel transfer, chunks are sent as soon as they are received, but unfortunately we think the overhead of this made no notable impact.
- Configuration 3 requires every chunk to pass through the full chain sequentially, it's quite a bit slower from the rest.

2.2.2 Session Restoration

These tests sees that a killed app restores all game state correctly, map data, player list, role, game config, and replay progress.

State	Steps	Output	Result
Lobby	1. Join a lobby, get accepted. 2. Force kill the app. 3. Reopen.	No session saved.	pass
Starting (map transfer)	1. Host starts game. 2. Force kill joiner's app during "Starting game..." while map is transferring. 3. Reopen.	No session saved. Game continues as normal for other players.	pass

Active game	1. Game started, GPS tracking active. 2. Force kill the app. 3. Reopen. 4. Select resume game.	Session saved. Resumes game. Map loads correctly with correct zones closed. Role matches what it was before kill. Time elapsed is accurate. GPS tracking resumes (notification reappears). Replay recording continues from previous frame count.	pass
Finished game	1. Game has ended. Replay not yet exchanged. 2. Force kill the app. 3. Reopen. 4. "Resume Offline Game".	Session saved. Resumes to finished game state. Map loads correctly. Final elapsed time preserved. Player list with roles intact. Replay exchange can proceed normally.	pass

2.3 Disposability

It is important that when a user exits from one screen to another in the app, services in the screen are correctly destroyed. When a player leaves a game at any point, all resources are properly cleaned up, GPS tracking stops, Nearby Connections or socket connections close, timers cancel, and no crashes occur on return to the main menu.

No Android foreground service notification, if persists, the LocationService wasn't disposed of.
No Nearby Connections activity (no advertising/discovering visible to other devices).
No crash on the main menu.
No continued log output in the debug console.

Offline Disposability Tests

State	Steps	Output	Result
Lobby (host)	1. Host creates lobby. 2. One player joins. 3. Host presses back and disbands game	Return to main menu. No session saved. Nearby advertising stops. Nearby connections closed.	pass
Lobby (joiner)	1. Join a lobby, get accepted. 2. Press back, confirm leave.	Returns to main menu. No session saved. Lobby sees joiner leave. Nearby connections closed.	pass
Lobby (pending)	1. Join a lobby, waiting for acceptance. 2. Press back, confirm leave.	Returns to main menu. No session saved. Host sees pending player disappear. Nearby connections closed.	pass
Lobby (kicked)	1. Join a lobby, get accepted. 2. Host kicks the player.	Returns to main menu. No session saved. Lobby sees joiner leave. Player sees "You have been kicked by the host" snackbar. Nearby connections closed.	pass
Starting (map transfer)	1. Host starts game. 2. Joiner presses back during "Starting game..." while map transfer	Returns to main menu. No session saved. Chunked transfer interrupted without crash. Nearby connections closed.	pass
Active game	1. Game started, GPS tracking active. 2. Press back, confirm leave.	Returns to main menu. Session saved. GPS notification disappears.	pass
Active game (force kill)	1. Game started, GPS tracking active. 2. Force kill the app from Android recents. 3. Reopen.	Session saved. Tapping resume restores game session correctly (including map and replay data). No left over GPS service.	pass

Finished game (pre-replay)	1. Game ended, replay not yet exchanged. 2. Press back, confirm leave.	Returns to main menu. Session saved. Tapping resume restores game session correctly (including map and replay data).	pass
Finished game (replay exchange)	1. Game ended. Non-host presses "Send Replay Data". 2. While discovering or transferring, press back, confirm leave.	Returns to main menu. Session saved. Tapping resume restores game session correctly (including map and replay data). No crash from pending discovery callbacks. Nearby connections closed.	pass
Finished game (post-replay)	1. Game ended, replay assembled and received. 2. Press back confirm leave.	Leaves immediately (no confirmation dialog since replay is complete). Session saved.	pass

Online Disposability Tests

State	Steps	Output	Result
Lobby (host)	1. Host creates lobby. 2. One player joins. 3. Host presses back and disbands game	Return to main menu. Session cleared. All players see "Game disbanded" snackbar. Socket disconnected.	pass
Lobby (joiner)	1. Join a lobby, get accepted. 2. Press back, confirm leave.	Returns to main menu. Session saved. Server removes player from game. Other players see player left. Socket disconnected.	pass
Lobby (pending)	1. Join a lobby, waiting for acceptance. 2. Press back, confirm leave.	Returns to main menu. No session saved. Server cleans up pending request. Host sees pending player disappear. Socket disconnected.	pass
Lobby (kicked)	1. Join a lobby, get accepted. 2. Host kicks the player.	Returns to main menu. No session cleared Player sees "You have been kicked by the host" snackbar. Other players see player left. Socket disconnected.	pass
Starting (map transfer)	1. Host starts game. 2. Joiner presses back during "Starting game..." while map transfer	Returns to main menu. Session saved. Server marks player as disconnected (not removed). Other players see "disconnected" status. Socket disconnected.	pass

Active game	1. Game started, GPS tracking active. 2. Press back, confirm leave.	Returns to main menu. Session saved. Server marks player as disconnected (not removed). Other players see "disconnected" status. Socket disconnects. GPS notification disappears.	pass
Active game (kicked)	1. Game started, GPS tracking active. 2. Host kicks the player.	Returns to main menu. No session cleared Player sees "You have been kicked by the host" snackbar. Other players see player left. Socket disconnected.	pass
Active game (force kill)	1. Game started, GPS tracking active. 2. Force kill the app from Android recents. 3. Reopen.	Session saved. Tapping resume restores game session correctly (including map and replay data). Server marks player as disconnected (not removed). Other players see "disconnected" status. Socket disconnects. No left over GPS service.	pass
Finished game (pre-replay)	1. Game ended, replay not yet exchanged. 2. Press back, confirm leave.	Returns to main menu. Session saved. Tapping resume restores game session correctly (including map and replay data). Server marks player as disconnected (not removed). Other players see "disconnected" status. Socket disconnects.	pass
Finished game (post-replay)	1. Game ended, replay assembled and received. 2. Press back confirm leave.	Leaves immediately (no confirmation dialog since replay is complete). Session saved. Other players see "disconnected" status. Socket disconnects.	pass

Changes Made:
Recording each dispose scenario with entries with written entries made them easy to track. In particular, it was important to know when the session was saved and cleared. We generally made sessions save all the time and rarely clear because it's useful and mostly harmless, but it could be confusing. Maybe user testing should’ve been done in this area.

3. Manual Integration Testing

3.1 Online Connectivity

These tests helped us confirm everything in online mode was in working order on all sides, client and server.

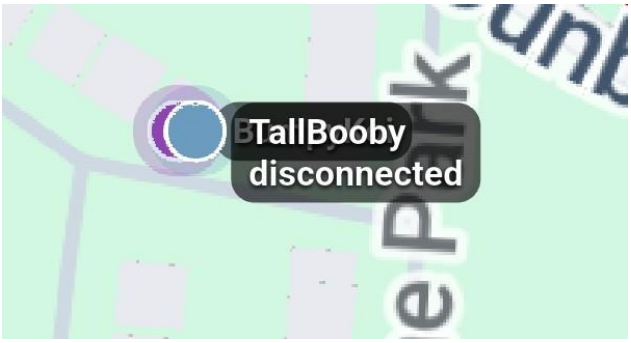
3.1.1 Connection Loss

Tests 1: Network Loss During Active Phase

Steps	Description
1. Online game has begun, in active phase. 2. Primary client turns on airplane mode for 60 seconds. 3. Primary client turns off airplane mode and reconnects.	This test sees if each role is correctly recognising when a client has disconnected.

Output (of every role)

Primary Client	Server	Other Clients
2. Cloud icon changes 2. Continually retries connection 3. Reconnects smoothly, latest player role updates and game status retrieved and player movement begins being sent and retrieved again.	2. After ping timeout (30s), primary client marked status property in corresponding Player object as “disconnected”. Broadcasts include this info. 3. Once reconnected, Player object is updated and broadcasts begin to reflect this.	2. Appears as "disconnected" status on map and player list. 3. Appears active again.



3.1.1 Tests 1

Tests 2: GPS Loss During Active Phase

Steps	Description
1. Online game has begun, in active phase. 2. Primary client turns off GPS functionality for 60 seconds. 3. Primary client turns on GPS functionality and reconnects.	This test sees if each role is correctly recognising when a client is still connected but not releasing GPS updates.

Output (of every role)

Primary Client	Server	Other Clients
2. GPS stream remains active. 2. Continually retries for new GPS output every 5 seconds. 2. Stays ingame. 3. Reconnects smoothly, begins sending GPS updates again.	2. After staleThresholdMs (15s), primary client marked status property in corresponding Player object as stale. Broadcasts include this info. 3. Once reconnected, Player object is updated and broadcasts begin to reflect this.	2. Has “last seen Xs ago” label on map. Accurately reflects timestamp of last seen. 3. Appears active again.



3.1.1 Tests 2

3.1.2 Session Resumption

Tests 1: Reconnection To Lobby After App Kill

Steps	Description
1. Online game in lobby phase. Primary client is a joiner who has been accepted. 2. Primary client force kills the app from Android recents. 3. Primary client reopens the app and taps "Reconnect".	This test sees that a joiner can reconnect to a lobby they were in before the app was killed. Since leaving a lobby as a non-host calls leaveGame, the saved session is used to rejoin the same game code as a fresh join.

Output (of every role)

Primary Client	Server	Other Clients
3. Rejoins lobby as a new player.	2. Player explicitly removed due to socket dropping. (transport closed) 3. New join request received.	2. Disappears from player list. 3. Reappears in player list as new player after host accepts.

```
› 11:33:48 [disconnect] Client: (djKql9sfS2tZaW4eAAAt) Reason: (transport close) PlayerName: (UptightSquid) PlayerId: (124606b6-9a99-4c41-9ec5-5e04063f8e
           a3) "fully removed from lobby game" (74H8RP)
```

3.1.2 Tests 1: Server log

Tests 2: Session Restoration After App Kill

Steps	Description
1. Online game has begun, in active phase. 2. Primary client force kills the app from Android recents. 3. Primary client reopens the app and taps the "Reconnect" button on the main menu. 4. Host ends the game.	This test sees that a killed app restores all game state correctly, map data, player list, role, game config, and replay progress.

Output (of every role)

Primary Client	Server	Other Clients
3. Rejoins game. Map loads correctly with correct zones closed. Role matches what it was before kill. Time elapsed is accurate. GPS tracking resumes (notification reappears). Replay recording continues from previous frame count. 4. Game finishes. Replay auto-submits with frames from both sessions combined. Logs show correct total frame count.	2. Player explicitly removed due to socket dropping. (transport closed) 3. Rejoin event received. Player marked as connected. Broadcasts resume including this player. 4. Replay received from primary client. Total frame count includes pre-leave and post-reconnect frames.	2. Disappears from player list. 3. Reappears active again. 4. Game finishes normally.

4. Manual Functional Testing

Functional testing was vital to ensuring the completeness of Tag Battle Royale. We captured requirements in the form of user stories, containing functionality from the perspective of a user. Throughout development we continuously defined and refined these user stories. Near the end of development we ensured that all functionality contained by the users stories was implemented and consistent. In the end we had 73 user stories with all of them achieving a pass result. We group the user stories in three categories:

4.1 Maps

The map editor is the first step on the user's journey. Its end result is used by all other sections of the application. Here it's decided where the boundaries for play are. We ensured its functionality from the following user stories:

Map Editor				
User Story	**Input**	**Expected Output**	**Actual Output**	**Result**
User can create map	1. User navigates to map editor. 2. User presses create new map. 3. User selects where to play using the map embed. 4. User captures an image of the area. 5. User draws zones on the image. 6. User names the zones and the map. 7. User saves the map.	New map is saved with the user-defined image, zones, zone names, and map name.	New map is saved with the data defined by the User	pass
User can select where to play using a maps embed	1. User navigates to map editor. 2. User presses create new map. 3. User uses the map embed to move to the desired area.	The map embed navigates to the selected area.	The selected area is navigated to	pass
User can capture image of where to play	1. User navigates to map editor. 2. User presses create new map. 3. User selects where to play. 4. User captures the image of the selected area.	The selected map area is captured as an image for editing.	The selected area is captured	pass
User can draw on image to define zones	1. User opens the map editor with a captured map image. 2. User draws zone boundaries on the image.	Drawn strokes appear on the image and define zone boundaries. Faces are defined	The faces are found and the zone bounds are drawn	pass
User can use finger to draw curves with pen tool	1. User opens the map editor. 2. User selects the pen tool. 3. User drags their finger across the image.	Curved freehand lines are drawn on the image following the user's input. Faces are defined	The faces are found and the zone bounds are drawn	pass
User can use straight line tool for straight lines	1. User opens the map editor. 2. User selects the straight line tool. 3. User places line points on the image.	Straight line segments are drawn between the selected points. Faces are defied	The faces are found and the zone bounds are drawn	pass
User can use eraser to erase strokes	1. User opens the map editor. 2. User draws on the image. 3. User selects the eraser tool. 4. User erases part of a stroke.	The erased stroke segment is removed from the image.	The stroke is removed from the faces	pass
User can undo previous action	1. User performs an action in the map editor. 2. User presses undo.	The most recent action is reversed.	Action is reversed including redo, straightline tool, pen tool and eraser tool	pass
User can redo undo action	1. User performs an action in the map editor. 2. User presses undo. 3. User presses redo.	The previously undone action is restored.	undo action is undone	pass
User can save map	1. User creates a map in the map editor. 2. User presses save.	The map is saved and can be accessed later.	Map is saved	pass
User can name map	1. User creates a map. 2. User enters a name for the map.	The map name is stored with the map.	Map name is saved	pass
User can name zones in map	1. User creates a map with zones. 2. User exits draw mode 3. User selects a zone. 4. User enters a name for the zone.	The zone name is stored with that zone.	Zone name is saved	pass
User can view created map	1. User navigates to saved maps. 2. User selects a created map.	The selected saved map is displayed.	Map can be viewed	pass
User can delete map	1. User navigates to saved maps. 2. User selects a map. 3. User presses delete. 4. User confirms deletion.	The selected map is removed from saved maps.	Map is deleted	pass
When a user deletes a map any replays for that map are deleted	1. User navigates to saved maps. 2. User deletes a map that has replay files linked to it.	The selected map is deleted and all replay files linked to that map are also deleted.	Map and replays are deleted	pass

4.2 Game

Next is lobby functionality. We sectioned the gane user stories into their respective phases. We start with the overall lobby functionality, then the lobby phase functionality, next the starting phase functionality, the active phase functionality and finally the finished phase functionality.

Lobbies				
User Story	**Input**	**Expected Output**	**Actual Output**	**Result**
User can host online game	1. User navigates to the online screen. 2. User selects host online game.	An online lobby is created and the user becomes the host.	Lobby is hosted	pass
User can host offline game	1. User navigates to the offline screen. 2. User selects host offline game.	An offline lobby is created and the user becomes the host.	Lobby is hosted	pass
User can join online game with a code	1. User navigates to join online game. 2. User enters a valid game code. 3. User confirms join.	The user joins the corresponding online lobby and can be accepted by the host.	User joins and waits for host to accept them	pass
User can join nearby offline game	1. User navigates to join offline game. 2. User selects a nearby lobby from the discovered list.	The user joins the selected nearby offline lobby and can be accepted by the host.	User joins and waits for host to accept them	pass

- **Lobby Phase**

  Here we have the lobby phase. In this section we set up the lobby for the game we were about to play. This includes actions from kicking players to changing the settings and selecting the map.

Lobby phase				
User Story	**Input**	**Expected Output**	**Actual Output**	**Result**
Host can change lobby settings	1. Host enters the lobby. 2. Host changes one or more lobby settings.	The updated lobby settings are updated	settings are updated	pass
Host can select map to play in and zone close order for that map	1. Host enters the lobby. 2. Host selects a map. 3. Host sets the order that zones will close.	The selected map and zone close order are stored for the game.	Map and zone close order are stored	pass
Host can accept or deny joining players before they join	1. A player requests to join the lobby. 2. Host reviews the join request. 3. Host accepts or denies it.	The player is either admitted to the lobby or refused entry.	Host can accept or deny joining players	pass
Host can kick player	1. Host enters the lobby. 2. Player joins lobby 3. Host selects a player. 4. Host chooses kick.	The selected player is removed from the lobby.	player is kicked	pass
Player can send messages in all chat	1. Player opens all chat in the lobby. 2. Player types a message. 3. Player sends the message.	The message is sent to the all chat channel.	Message is sent	pass
Players receive messages in all chat	1. A player sends a message in all chat. 2. Other players are in the lobby.	All players in the lobby receive and can view the message in all chat.	Mesage is recieved	pass
Host can start game	1. Host is in the lobby with at least one runner. 2. Host selects map and close order. 3. Host presses start game.	The game begins for all players in the lobby and the starting phase initiates	The game starts	pass
(online) Player who has left can quickly connect back from main menu screen	1. Player joins a lobby. 2. Player disconnects from the session. 3. Player reconnects to the same lobby from main menu.	Player can quickly reassume their position in a game they left	Player will be treated as a new player	pass

- **Starting phase**

  The starting phase is responsible for synchronizing information between players. In this step the map and other important bits of information are sent from the host to the players. Additionally, services like the replay recording start.

Starting Phase				
User Story	**Input**	**Expected Output**	**Actual Output**	**Result**
Map, game settings and zone close order is sent to all players in game	1. Host starts the game.	All players receive the selected map data and other data for the game and return ack	Map and others is recieved and ack returned	pass
Grace period starts	1. Host starts the game.	The game enters the grace period state.	Grace period starts	pass
Runners see banner to run away during grace period	1. Game enters grace period. 2. A player has the runner role.	The runner sees a banner instructing them to run away.	Banner is displayed	pass
Chasers see banner to stay in place during grace period	1. Game enters grace period. 2. A player has the chaser role.	The chaser sees a banner instructing them to stay in place.	Banner is displayed	pass
All players see time until grace period ends	1. Game enters grace period.	A countdown showing the remaining grace period time is displayed to all players.	Countdown is displayed	pass
Map and zones are displayed	1. Game starts.	The map and its zones are displayed to each player.	Map is displayed	pass
Players can only see their own position during grace period	1. Game is in grace period. 2. Player views the map.	Each player can see only their own position and not the positions of others.	Only position of user is drawn	pass
Runners cannot become chasers during grace period	1. Game is in grace period. 2. Runner holds become chaser button.	The runner remains a runner during the grace period.	Runner's role does not switch	pass
Players positions are recorded for the replay	1. Game starts. 2. Replay recording starts.	Player position data is recorded over time for replay use.	Replay data is recorded	pass

- **Active phase**

  The active phase is where the main gameplay of Tag Battle Royale is contained. Here we need to display any relevant information to the user like players they're allowed to see, the zones that have closed and when the next zone will close for example. Some of the functionality in this section is online or offline only shown by either.

Active Phase				
User Story	**Input**	**Expected Output**	**Actual Output**	**Result**
(online) Player can send messages in all chat	1. Player opens all chat in the lobby. 2. Player types a message. 3. Player sends the message.	The message is sent to the all chat channel.	Message is sent	pass
(online) Players receive messages in all chat	1. A player sends a message in all chat. 2. Other players are in the lobby.	All players in the lobby receive and can view the message in all chat.	Mesage is recieved	pass
(online) Only runners can send messages in runner chat	1. Runner opens runner chat. 2. Runner types and sends a message.	The message is sent in runner chat. Non-runners cannot send messages there.	Message is only seen by runners	pass
(online) Only runners receive messages in runner chat	1. A runner sends a message in runner chat.	Only runners receive and can view the message in runner chat.	Only Runnenrs see the message	pass
(online) Only chasers can send messages in chaser chat	1. Chaser opens chaser chat. 2. Chaser types and sends a message.	The message is sent in chaser chat. Non-chasers cannot send messages there.	Message is only seen by chasers	pass
(online) Only chasers receive messages in chaser chat	1. A chaser sends a message in chaser chat.	Only chasers receive and can view the message in chaser chat.	Only Chasers see the message	pass
Zones will close in the order set by the host	1. Host has selects zone close order before the game. 2. The game starts.	Zones close in the order defined by the host.	Zones close in correct order	pass
(online) Host can change the next zone to close	1. Game starts. 2. Host selects a different zone as the next one to close.	The next zone to close is updated.	zone close order is updated correctly	pass
Chasers can see next zone to close	1. Game has started. 2. Player is a chaser.	The chaser can see which zone will close next.	Next zone to close is shown	pass
All players can see closed zones on the map	1. One or more zones have closed during the game.	Closed zones are visibly marked on the map for all players.	Closed zones are displayed	pass
Runners cannot see the next zone to close	1. Game has started. 2. Player is a runner.	The runner cannot see which zone will close next.	Next zone to close is not displayed	pass
All players can see the time until the next zone closes	1. Game has started.	A countdown showing time until the next zone closes is displayed to all players.	Countdown to zone close is displayed	pass
(online) Chasers can see runners on the map when the runners are outside an open zone	1. Game has started. 2. A runner is outside an open zone. 3. A player is a chaser.	The chaser can see that runner's location on the map.	Runners are displayed when outside open zones	pass
(online) Chasers cannot see runners on the map when the runners are inside an open zone	1. Game has started. 2. A runner is inside an open zone. 3. A player is a chaser.	The chaser cannot see that runner's location on the map.	runners are not shown when inside an open zone	pass

Runners cannot see chasers on the map	1. Game has started. 2. Player is a runner.	The runner cannot see chaser locations on the map.	chasers and other runners are not displayed to runners	pass
Runners can turn into chasers	1. Game has started. 2. A runner holds the i was caught button.	The runner's role changes to chaser.	runner becomes chaser	pass
(online) When all runners are chasers the game ends	1. Game has started. 2. The final remaining runner becomes a chaser.	The game ends automatically.	Game ends	pass
(online) The host can end the game	1. Game is in progress. 2. Host presses end game.	The game ends for all players.	Game ends	pass
(online) When a player sees another player on the map, they can also see how old that location is.	1. A player is visible on the map to another player.	The visible player's displayed location includes an indication of how old that location data is.	the locations age is displayed in seconds	pass
(offline) all players can locally end thier game	1. players play an offline game. 2. players can end the game for themself and go to the end screen	end screen is shown	end screen is shwon	pass
Player who has left can quickly connect back from main menu screen	1. Player joins a lobby. 2. Player disconnects from the session. 3. Player reconnects to the same lobby from main menu.	Player can quickly reassume their position in a game they left	Players can resume their session	pass

- **Finished Phase**
  Finally, we have the finished phase where players synchronize their replay information and down the replay / map.

Finished Phase				
User Story	**Input**	**Expected Output**	**Actual Output**	**Result**
(online) Player caught last is displayed as the winner	1. Game ends after all runners become chasers.	The player caught last is displayed as the winner.	players are ranked in revese order they were caught	pass
(online) Players send their replay information to the Host	1. Game ends.	Each player's replay data is sent to the host.	players send there replay data	pass
(offline) Players reconnect and send replay information to the host	1. Game ends. 2. Each player presses the reconnect and send replay buttom.	players reconnect and send replay data	players reconnect and replay data is sent	pass
Host combines the replays information and zone closing into a replay file	1. Game ends. 2. Host receives replay data from players. 3. Host processes replay and zone closure data.	A complete replay file is created by the host.	Host creates replay file	pass
Host sends complete replay file to players	1. Host has created the replay file after the game ends.	The replay file is sent from the host to the players.	host sends completed replay file	pass
Players download the replay and map files	1. Players receive replay and map file transfer after the game ends.	The replay and required map files are downloaded to the players' devices.	replay is downloaded and map file is download if it doesnt already exist	pass

4.3 Replay Viewer

The replay viewer is the end of the user's journey through our application. Here we provide a retrospective on the game's the user has played in a video like format.

Replay Viewer				
User Story	**Input**	**Expected Output**	**Actual Output**	**Result**
Users can view replays of games they've played	1. User navigates to replay viewer. 2. User selects a saved replay.	The selected replay is opened and displayed.	replays are displayed	pass
Users can see zone closings in the replay	1. User opens a replay.	The replay shows when and where zones closed during the game.	zone closings are shown as occured in the game played	pass
Users can see the position off all others players during the game in the replay	1. User opens a replay.	The replay displays the recorded positions of all players over time.	all players are displayed at there location at a specific time during the game	pass
Users displayed on the replay have their name and role (chaser or runner) shown	1. User opens a replay.	Player markers in the replay display the player's name and role.	player roles are displayed including role changes	pass
Users can scroll through the replay	1. User opens a replay. 2. User drags or uses replay controls to move through time.	The replay position changes to the selected point in time.	replay positions are displayed correctly	pass
Users can toggle the nameplate for the players	1. User opens a replay. 2. User toggles nameplate display.	Player nameplates are shown or hidden.	nameplates are toggleable	pass
Users can pause and unpause the replay	1. User opens a replay. 2. User presses pause or play.	The replay pauses or resumes.	the replay can pause or resume	pass
Users can toggle displaying specific players in the replay	1. User opens a replay. 2. User selects which players to show or hide.	Only the selected players are displayed in the replay.	unselected players are no longer displayed	pass
Users can minimise the controls for the replay	1. User opens a replay. 2. User presses the minimise control button.	The replay controls are collapsed or minimised.	the controls are minimised	pass
Users can delete a replay	1. User navigates to saved replays. 2. User selects a replay. 3. User presses delete. 4. User confirms deletion.	The selected replay is removed from saved replays.	replay is deleted	pass

5. User Testing

5.1 Introduction

User testing was important to many aspects of Tag Battle Royale. By having people unfamiliar with the application interact with it, we gained a vital understanding of how our target users use Tag Battle Royale, which parts of the design were intuitive and more importantly where users would potentially be confused and get stuck. Additionally, these users provided a new perspective into our systems and confidence in different parts of the system's ability.

We had a group of five people within the age 21 - 25 engage in our user testing. These subjects were fellow COMSCI students, which assures not only their fresh perspective but also their ability to test and pressure the systems of our application.

5.2 Methods

Our user testing was structured as follows.

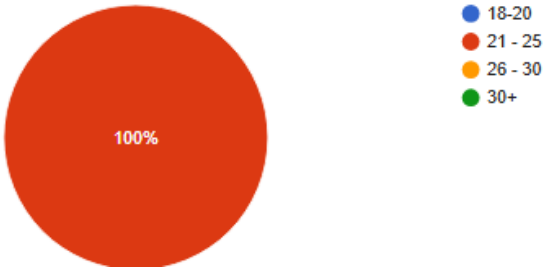
1. We provided all participants with the participant information sheet. To engage in our tests, each participant had to give their informed consent, by indicating their understanding of the study and signing their name.
2. Each participant was individually given the task "Create a map to play in". Researchers watched each participant use the map editor to create a map.
3. Next, they were tasked as a group to "Host and play an online game". The group would need to assign a person to host the game, select the map and zone closure order, accept other participants into the lobby and start the game. After starting the game the participants would follow the on screen prompts to play the game.
4. Again participants were instructed to play the game but this time offline. They engaged in similar activities as the previous step.
5. Finally, Each participant was individually instructed to "watch the replays of the games played". Researchers watched the participants engage with the replays.
6. Afterwards all participants were provided with the google forms link to evaluate Tag Battle Royale (see Appendix A).

5.3 Results

We conducted user testing with a sample group of 5 people. The Tag Battle Royale evaluation form contains the results of the user testing and can be broken into the following:

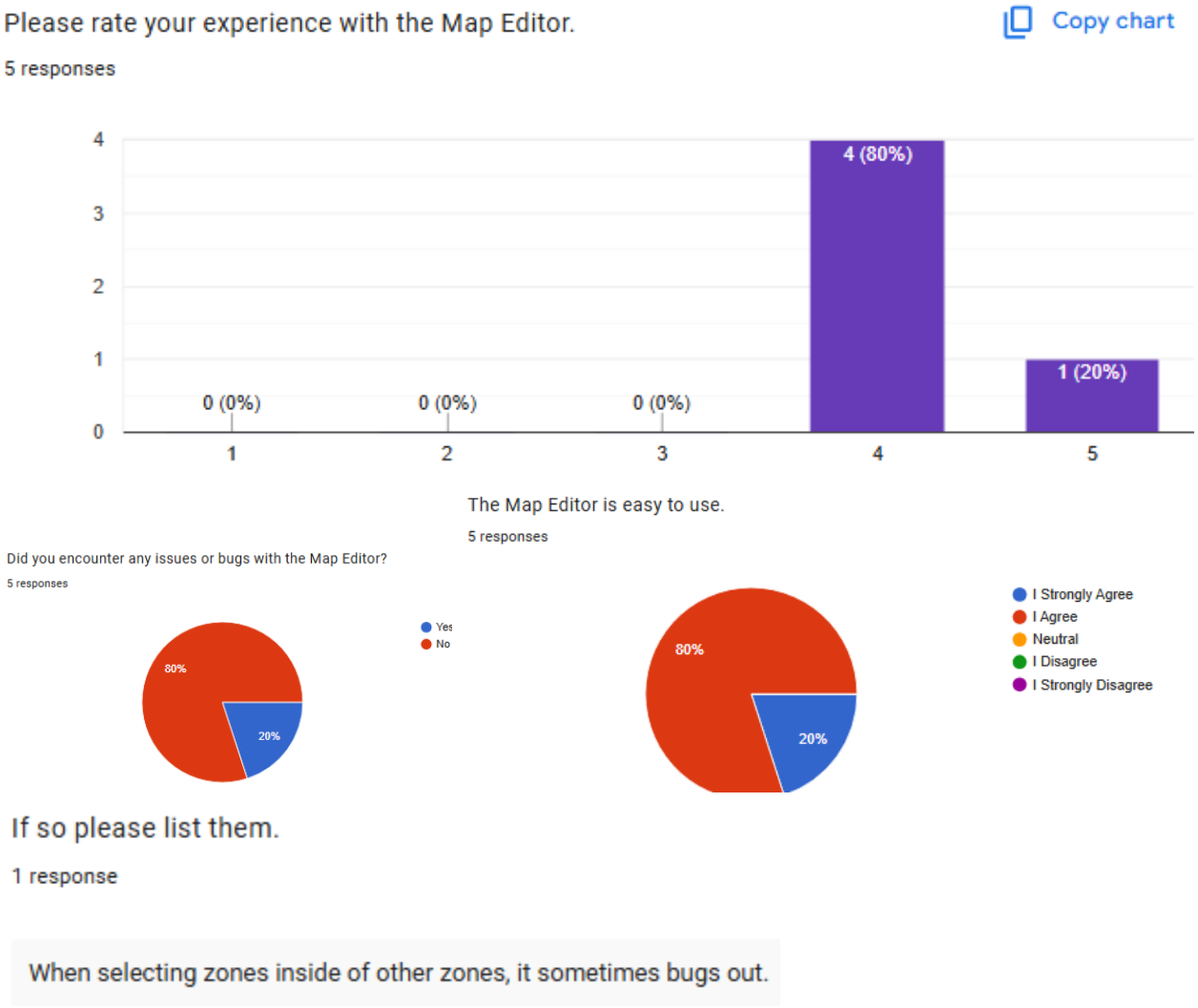
5.3.1 Participant Data

All participants of the user testing were within the age group 21-25



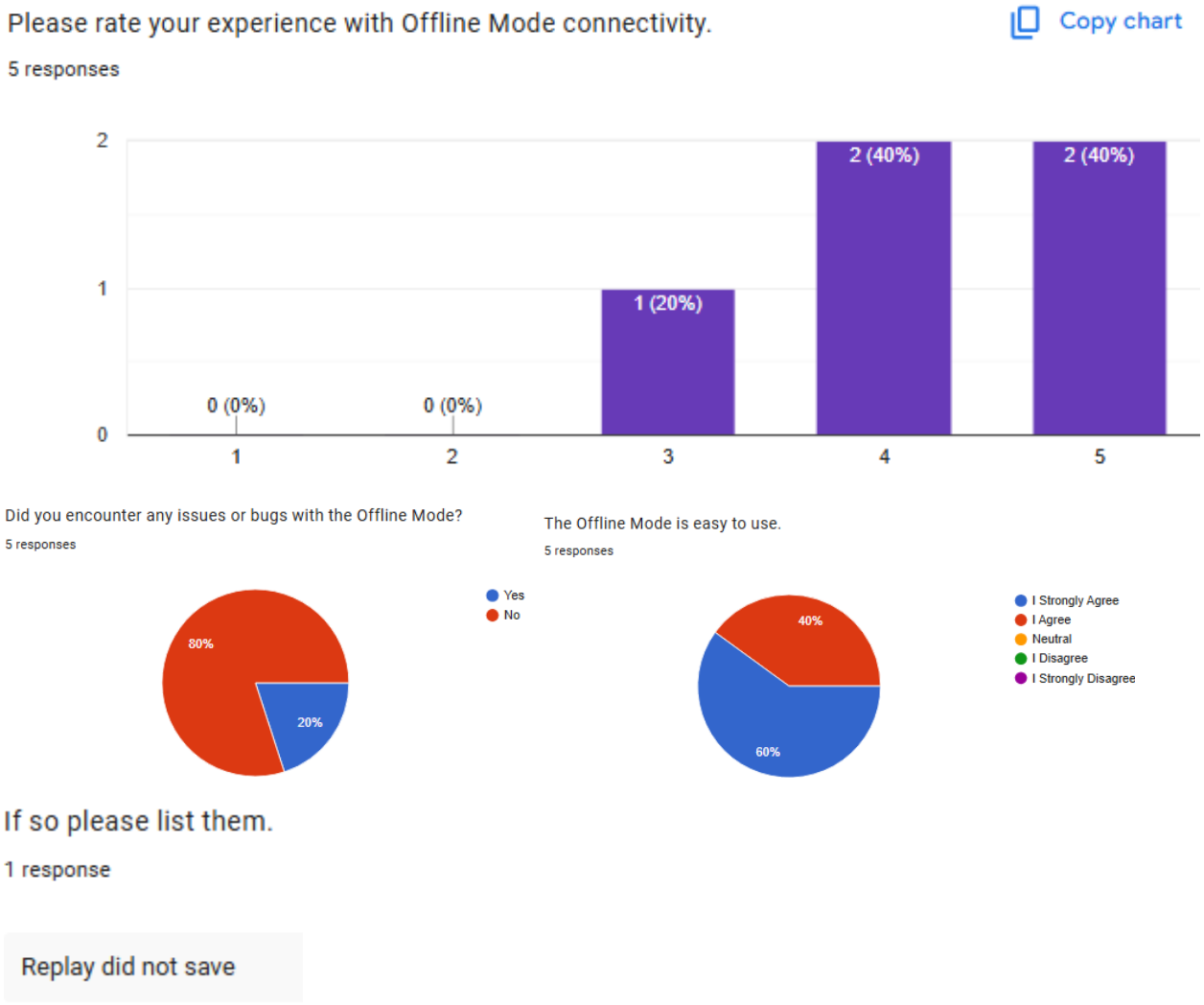
5.3.2 Map Editor

The Map Editor was ranked in three areas; Overall experience, ease of use and defect encounters. 80% of respondents rated the experience 4/5 and 20% rated the experience 5/5. For ease of use, 80% of respondents agreed and 20% of respondents strongly agreed with the claim “The Map Editor is easy to use”. 80% of participants did not encounter any defects in the Map Editor with 20% encountering a bug.



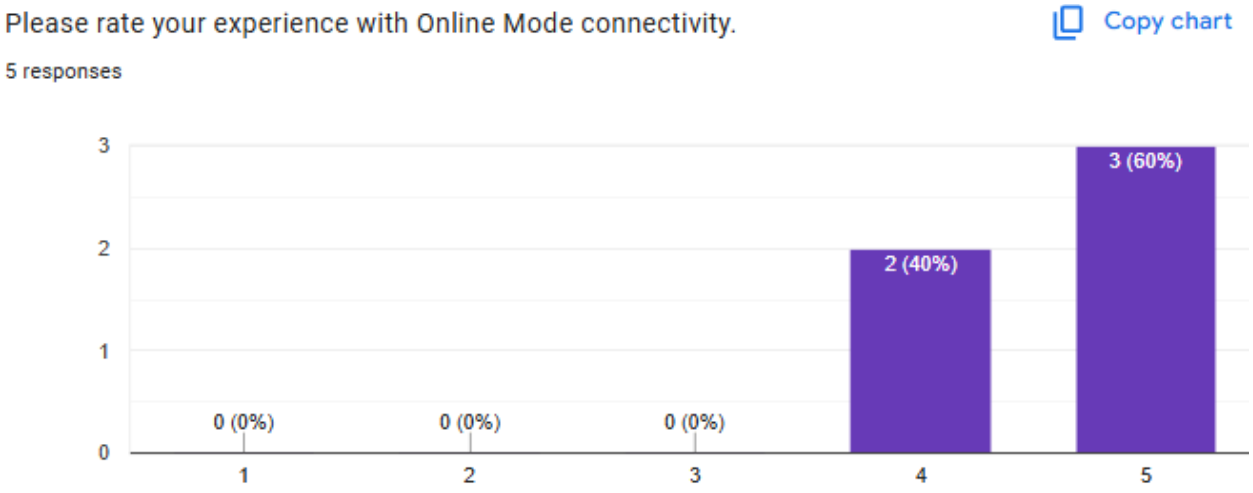
5.3.3 Offline Mode

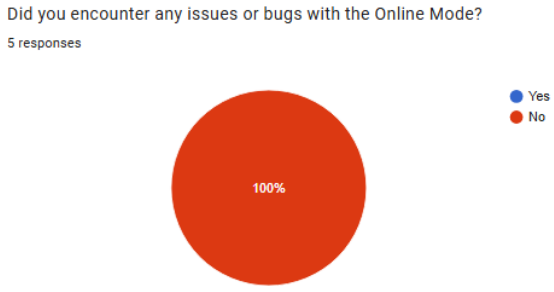
The Offline Mode was rated in three areas; Overall experience, ease of use and defect encounters. 20% of respondents rated the experience 3/5 ,40% of respondents rated the experience 4/5 and 40% of respondents rated the experience 5/5. 40% of respondents agreed and 60% of respondents strongly agreed with the claim “the Offline Mode is easy to use”. 80% of respondents encountered no bugs with 20% encountering a defect.

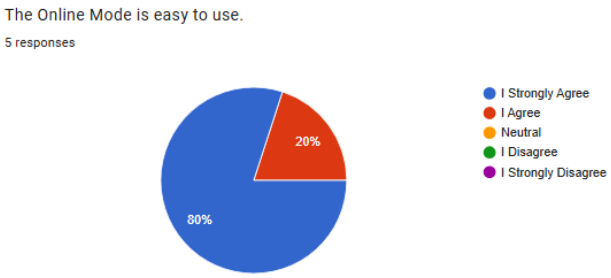


5.3.4 Online Mode

The Online Mode was rated in three areas; Overall experience, ease of use and defect encounters. 40% of respondents rated the experience 4/5 and 60% of respondents rated the experience 5/5. 20% of respondents agreed and 80% of respondents strongly agreed with the claim “The Online Mode is easy to use”. 100% of the respondents encountered no bugs with the Online Mode.

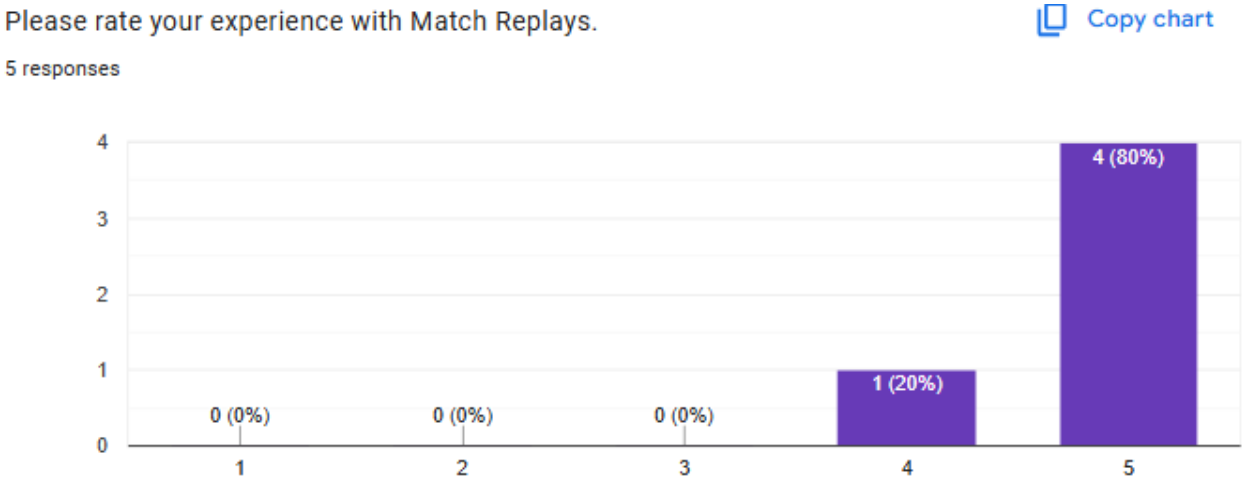


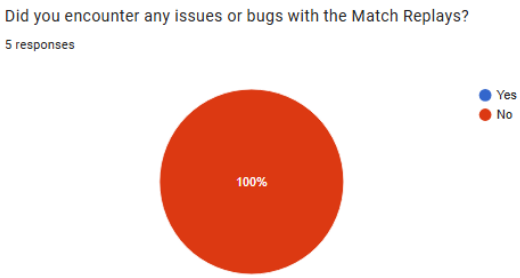


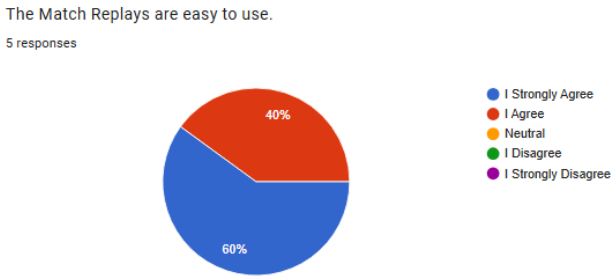


5.3.5 Replays

The Replay Viewer was rated in three areas; Overall experience, ease of use and defect encounters. 20% of respondents rated the experience 4/5 and 80% of respondents rated the experience 5/5. 40% of respondents agreed and 60% of respondents strongly agreed with the claim “The Match Replays are easy to use”. 100% of the respondents encountered no bugs with the Replays.

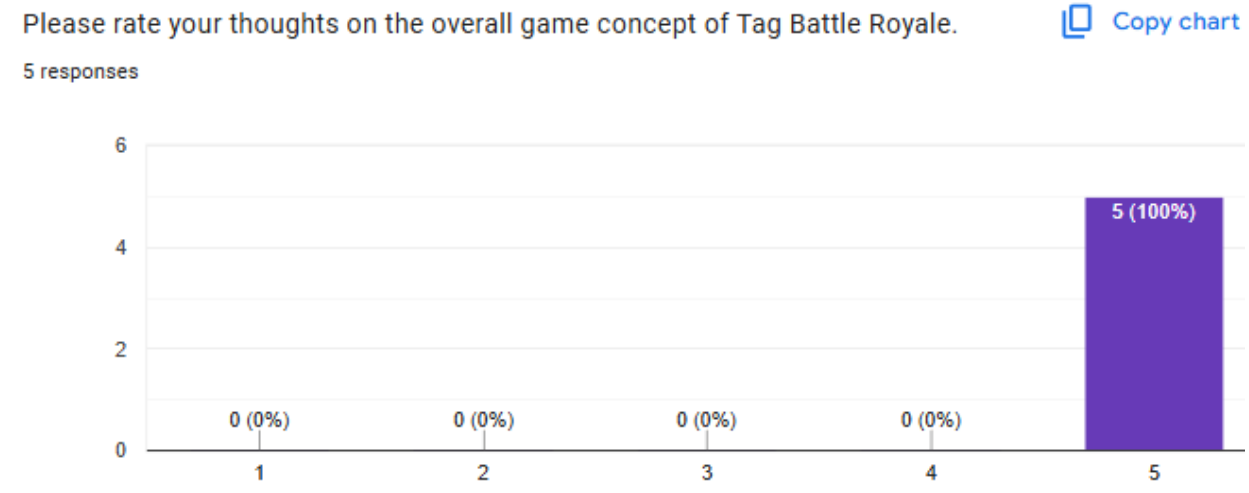


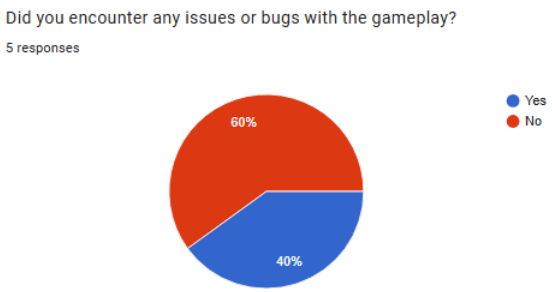


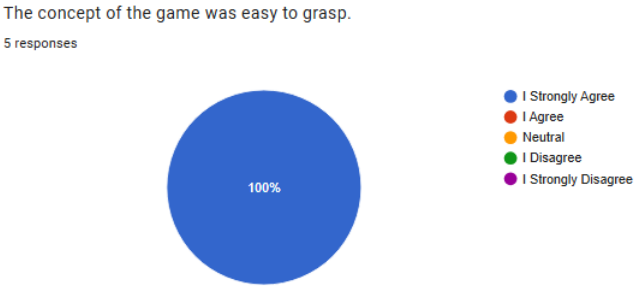


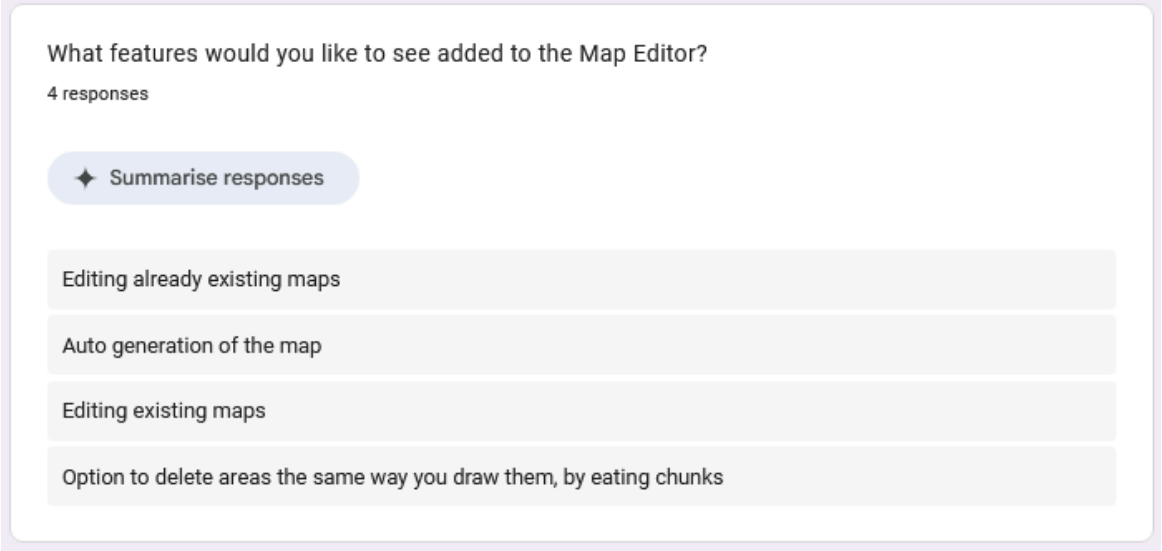
5.3.6 Overview

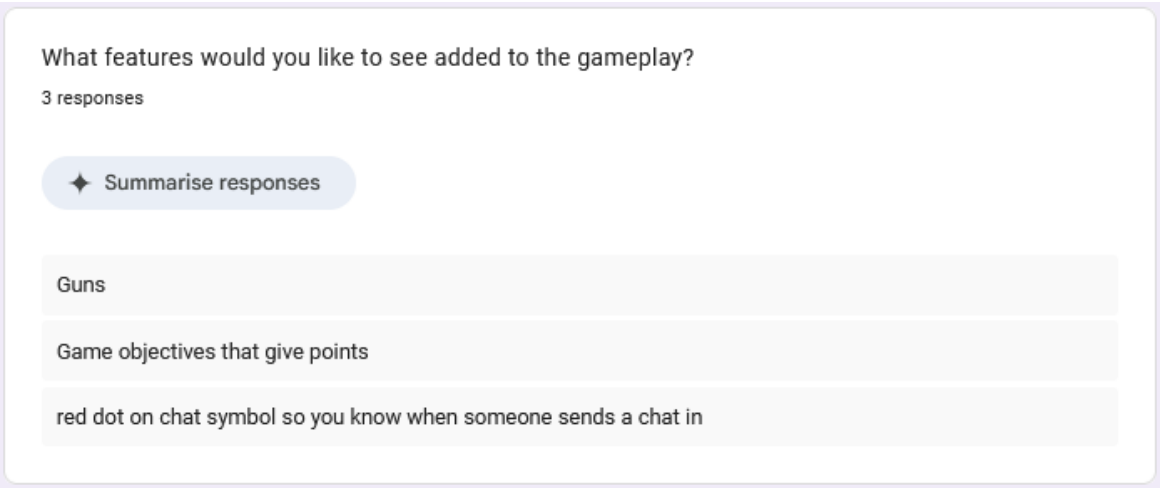
100% of the respondents liked the game Tag Battle Royale and found it easy to understand. Unfortunately 40% of respondents encountered a bug during user testing. Users also left feedback on possible future additions to the application.

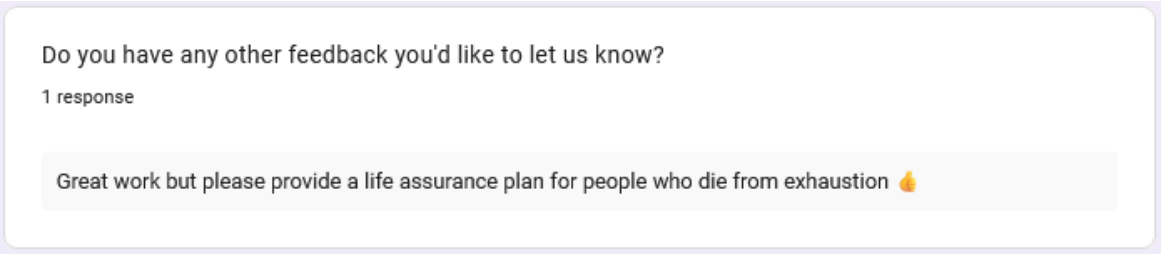












5.4 Conclusion

Tag Battle Royale received overwhelming high reviews in all four aspects of the application, the map editor, the offline mode, the online mode and the replay viewer. The user testing proved fruitful as a defect in the map editor and offline mode was found. The defect in the map editor was caused by an edge case that went undiscovered where two enclosed regions, one contained within the other, were connected by a single connection. These bridges break the face tracing logic and this issue is now handled by the face detection logic. This edge case has also been added to our automated testing to ensure the consistent full functionality of the system. The offline mode defect was caused by a bug present when a member leaving before all players had the replay. Participants showed interest in additional functionality, such as automatic map generation. This feedback was very interesting and has inspired potential future features for Tag Battle Royale.

Many important insights into the user experience with Tag Battle Royale were presented through user testing. Users were slow to onboard the application which highlighted the need of a tutorial or guide in the home page. Participants were overwhelmingly positive regarding Tag Battle Royale with interest in its public release. User testing was very important to our confidence in Tag Battle Royale and future user testing with a larger test group would be helpful.

6. Appendix

Appendix A: Tag Battle Royale evaluation form
Available from:
https://docs.google.com/forms/d/e/1FAIpQLSdi_eqUhywfiLyAacvEwfrppnQce7I5ptMJLHFmwY5Pktf6CQ/viewform?usp=sharing&ouid=104670999427580399133